

Verificação, Validação e Teste de Software (VVT) 2026.1

Day 5 - Conceitos básicos para a aplicação das atividades de VVT
Estratégia de teste

Prof. Dr. Awdren de Lima Fontão



UNIVERSIDADE
FEDERAL DE
MATO GROSSO DO SUL



Estratégia

- **stratos** = exército
- **agein / agós** = conduzir, guiar, liderar

Então, estratégia está ligada à ideia de:

“arte de conduzir forças” ou “arte de liderar ações para alcançar um objetivo”

um conjunto de escolhas orientadas a objetivos, realizado por atores que agem em contextos de restrição, oportunidade, incerteza e interação com outros atores.

As Múltiplas Facetas da Estratégia

a) Estratégia como ação orientada a fins



Um ator não faz tudo o que poderia fazer; ele **seleciona caminhos** para maximizar suas chances de alcançar algo.

b) Estratégia como adaptação ao contexto



A estratégia depende do ambiente. Ela não é universal. O que faz sentido em um contexto pode falhar em outro.

c) Estratégia como escolha sob restrição



Nenhum ator tem tempo, informação e recursos infinitos. Estratégia é, em parte, a arte de **decidir o que fazer e o que não fazer**.

d) Estratégia como articulação entre intenção e prática



Não basta ter um plano. Estratégia também envolve ajustar a ação conforme a realidade responde.

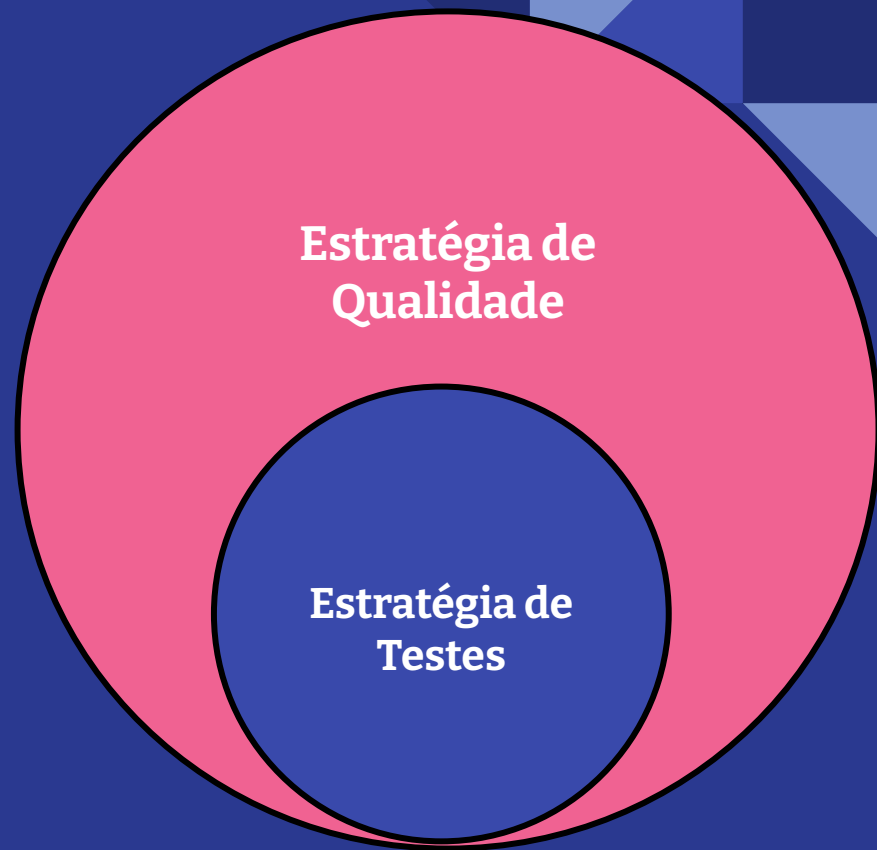
Estratégia em Engenharia de Software

Como a Engenharia de Software opera sob trade-offs permanentes (p.ex: prazo, custo, escopo, qualidade, risco e manutenibilidade), estratégia é o mecanismo pelo qual a equipe define prioridades e distribui esforços.

não há recursos infinitos, nem certeza total, nem uma única forma correta de agir. Por isso, a equipe precisa decidir conscientemente onde concentrar esforço, que práticas adotar e como equilibrar velocidade, custo, risco e qualidade.



Estratégia de testes



Estratégia de Teste

É a configuração sociotécnica de decisões, práticas, artefatos e interações por meio da qual o teste passa a ser realizado em um projeto; essa estratégia não é apenas planejada, mas emerge e evolui recursivamente a partir da interação entre condições técnicas, organizacionais e sociais.

On the Emergence of Testing Strategies:
A Socio-technical Grounded Theory

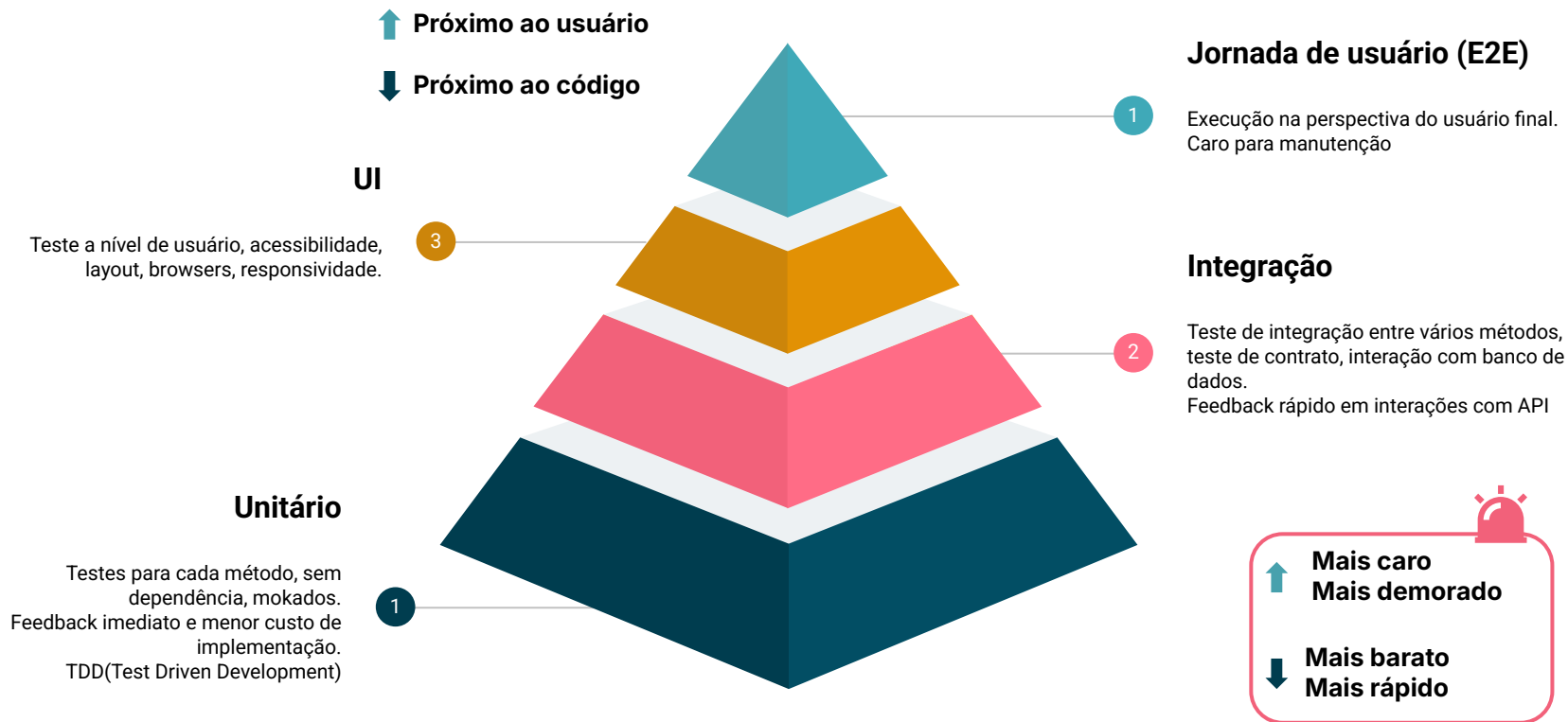
Mark Swillus · Rashina Hoda · Andy
Zaidman



Modelos de teste que podem ajudar a
visualizar a sua estratégia

Estratégias de teste

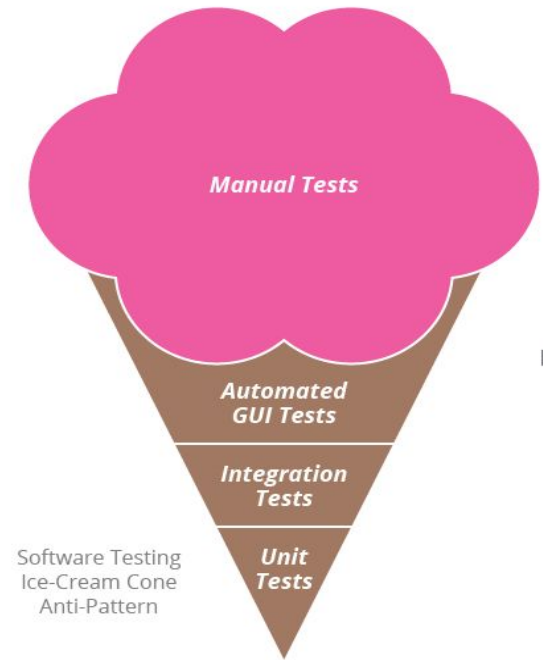
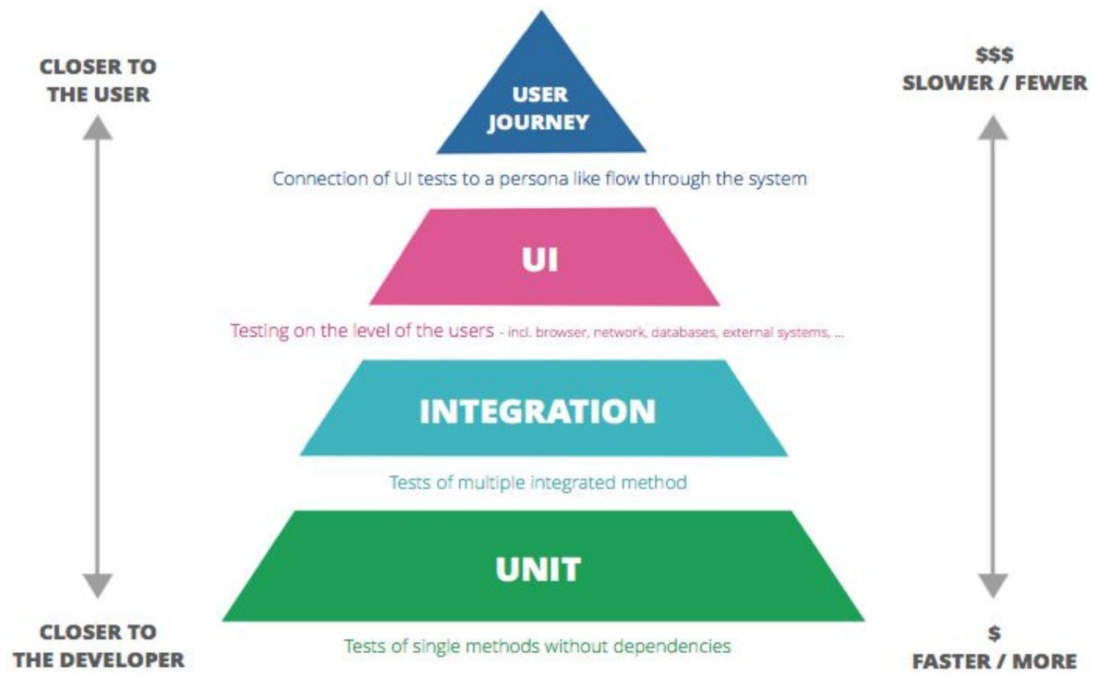
Pirâmide de teste: Fases de validação



*regra prática ou método orientador usado
para descobrir caminhos de solução, sem
garantir uma regra exata ou universal.*

A pirâmide de testes é uma heurística de balanceamento.
Ela ajuda a pensar como distribuir testes em diferentes
níveis para equilibrar custo, velocidade de feedback,
capacidade de diagnóstico e cobertura de riscos relevantes
ao usuário.

Eg:Automation test pyramid

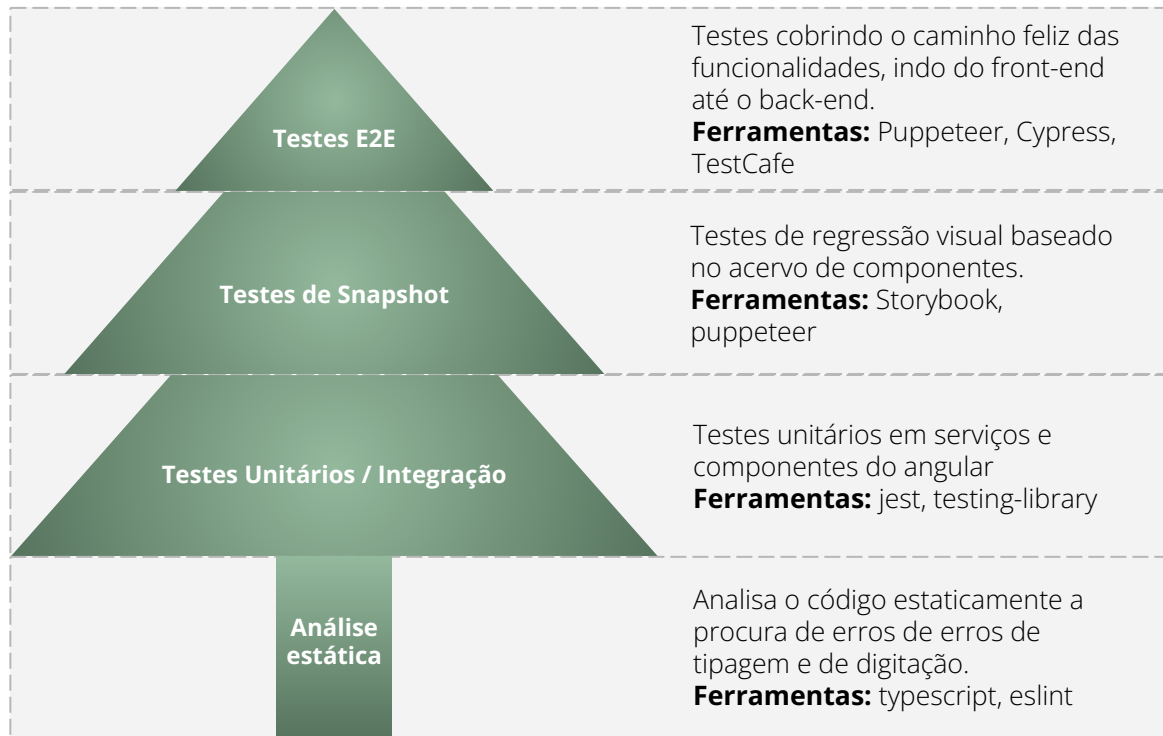


Antipadrão? antipattern?

Um antipadrão em Engenharia de Software é uma solução comum para um problema recorrente, mas que, em vez de trazer benefícios, causa efeitos negativos, como aumento da complexidade, dificuldade de manutenção ou baixa eficiência.

Os antipadrões geralmente surgem devido a más práticas, decisões mal informadas ou falta de planejamento adequado. Eles podem ocorrer em diversas áreas da Engenharia de Software, incluindo arquitetura, design, desenvolvimento e testes.

Pirâmide Árvore de testes



Com o avanço da tecnologia e de poder computacional, a pirâmide de testes vêm assumindo outros formatos.

A pirâmide em formato de árvore **evita a criação de testes sensíveis**, isto é, que quebrem quando há uma mudança em um detalhe de implementação de um componente de forma que seu comportamento seja mantido.

A árvore é uma heurística contemporânea para pensar portfólio de testes em sistemas modernos, especialmente quando há interfaces ricas, componentes reutilizáveis e ferramentas que permitem verificações mais granulares.

Imagine um sistema web. Se você quiser verificar tudo com E2E, você pode acabar criando testes que:

- clicam em tela;
- percorrem jornada completa;
- dependem de navegador, rede, back-end e dados;
- quebram por detalhes pequenos.

A árvore diz: **separe melhor as verificações**. Por exemplo:

- **análise estática** verifica erro de tipagem e problemas simples de código;
- **teste unitário / integração** verifica regra de negócio e interação entre partes;
- **snapshot / regressão visual** verifica se a interface mudou visualmente;
- **E2E** verifica só os fluxos mais críticos do usuário.

Modelo de queijo suíço

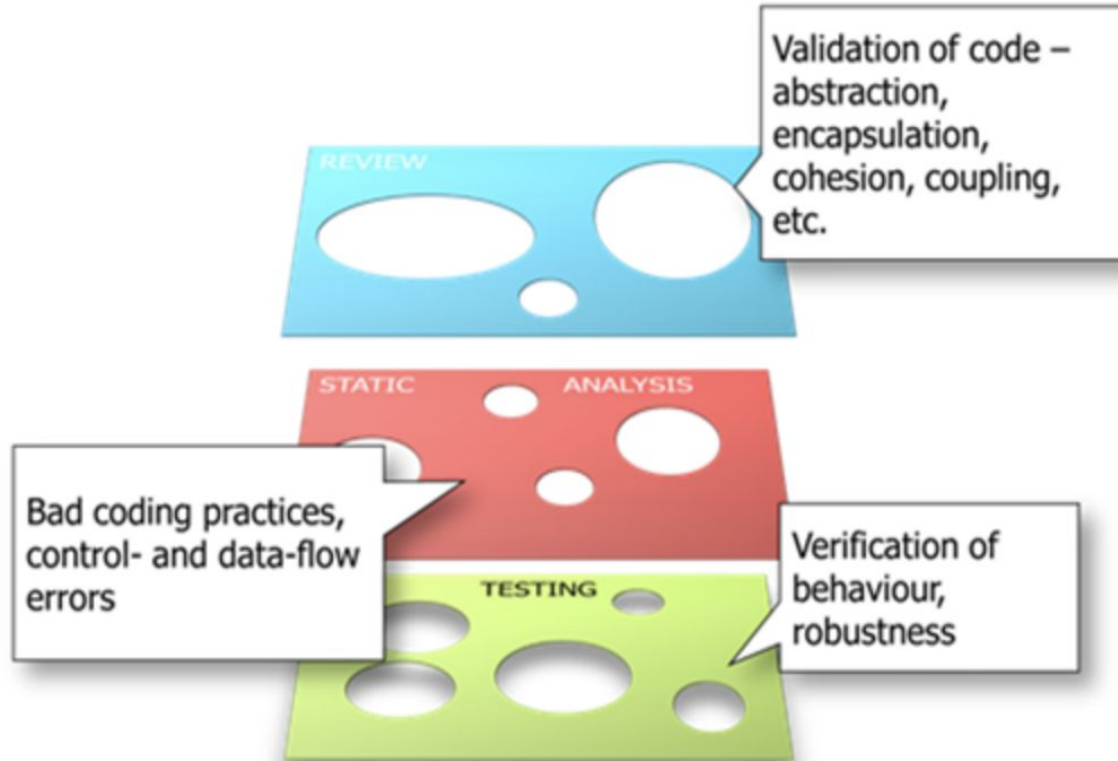
*Ou efeito da ação cumulativa.
Um sistema ou organização de erros isolados não destroem o
todo.*

Efeito acoplamento

Modelo de queijo suíço

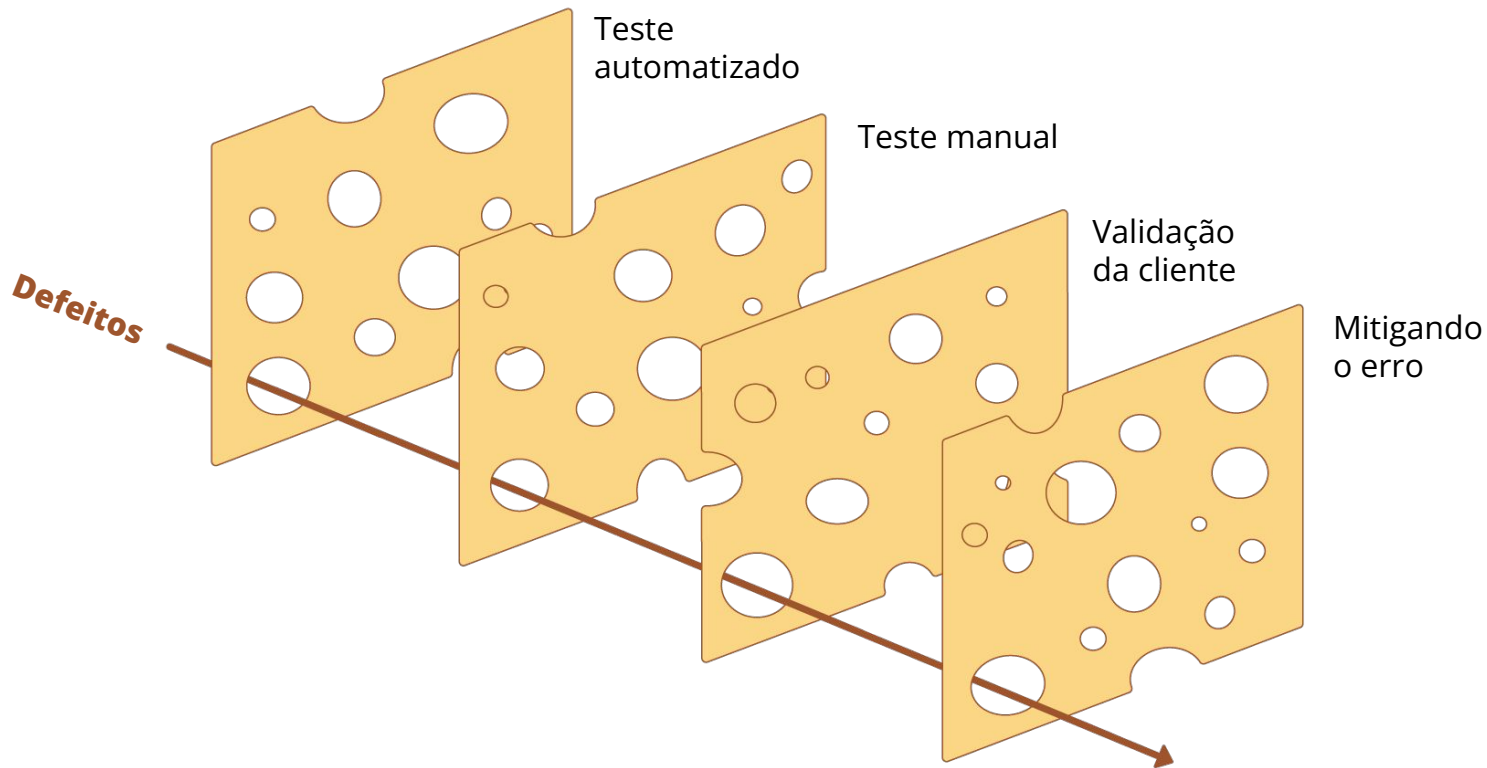
- Sistemas são construídos por camadas (fatias de queijo)
- Essas fatias não são perfeitas
- Elas têm imperfeições (buracos)
- Cada imperfeição é um possível problema em cada camada
- Se o alinhamento das camadas está de um jeito que permita que um defeito passe, o pior pode acontecer

Eg:Swiss cheese model

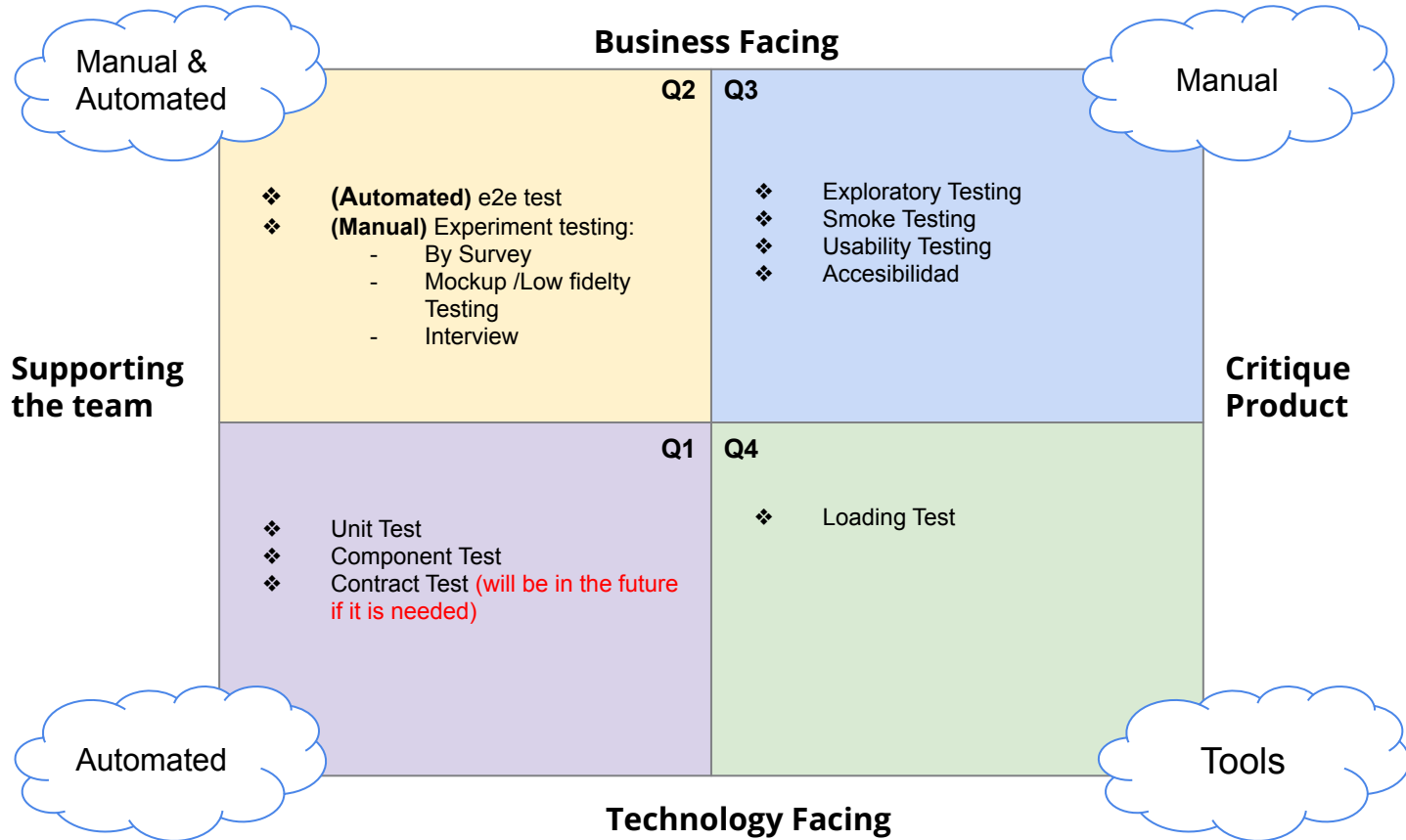


Modelo de queijo suíço

Processos



E.g: Automation Testing quadrant





“Em particular, as ferramentas de análise estática são incapazes de lhe dar confiança em sua lógica de negócios.

Os testes de unidade são incapazes de garantir que, quando você chama uma dependência, está chamando-a adequadamente.

Os testes de integração de interface do usuário são incapazes de garantir que você esteja passando os dados corretos para seu back-end e que você responda e analise os erros corretamente.

Testes de ponta a ponta são bastante capazes, mas normalmente você os executa em um ambiente de não produção ”

Kent C. Dodds

Toda estratégia
conta uma história.

Qual vai ser a sua?

DANA NORRIS

**STORYTELLING
NA PRÁTICA**

**10 REGRAS SIMPLES PARA
CONTAR UMA BOA HISTÓRIA**

ubook

E qual deve ser a base da nossa estratégia?

Eficaz & Sistemático

Ser eficaz significa focar na escrita dos testes certos. O teste de software envolve compensações: os testadores querem maximizar a quantidade de bugs encontrados enquanto minimizam o esforço necessário para encontrá-los. Como conseguimos isso? Sabendo o que testar.

Ser sistemático significa que, para um determinado código, qualquer desenvolvedor deve ser capaz de chegar à mesma suíte de testes. Muitas vezes, os testes são realizados de maneira ad hoc, onde os desenvolvedores criam casos de teste conforme vêm à mente. Isso resulta em diferentes test suites para o mesmo programa. Precisamos sistematizar nossos processos para reduzir a dependência da pessoa que está executando os testes.

Teste exaustivo é impossível

Não temos recursos para testar completamente nossos programas. Testar todas as possíveis situações em um sistema de software pode ser impossível, mesmo que tivéssemos recursos ilimitados.

Imagine um sistema de software com "apenas" 300 configurações ou flags (como no sistema operacional Linux). Cada flag pode ser verdadeira ou falsa (Boolean) e pode ser configurada independentemente das outras. O comportamento do sistema muda de acordo com a combinação dessas flags.

Com duas opções para cada uma das 300 flags, existem 2^{300} ($\approx 2,04 \times 10^{90}$) combinações possíveis que precisariam ser testadas. Para comparação, estima-se que existam 10^{80} átomos no universo. Ou seja, este sistema tem mais combinações a serem testadas do que o número de átomos no universo.

Sabendo que testar tudo é impossível, precisamos escolher e priorizar o que testar.

Saber quando parar



Priorizar quais testes desenvolver é um desafio. Criar poucos testes pode resultar em um sistema que não funciona como esperado (ou seja, cheio de bugs). Por outro lado, criar muitos testes sem critério pode gerar testes ineficazes, desperdiçando tempo e dinheiro.

O objetivo deve ser **maximizar** a detecção de bugs enquanto **minimizamos o custo desse processo**. Para isso, discutiremos no futuro critérios de adequação, que ajudarão a determinar quando parar de testar.

Variabilidade é importante (paradoxo do pesticida)



Nenhuma técnica isolada pode ser aplicada para encontrar todos os bugs. **Diferentes técnicas revelam diferentes tipos de falhas.** Se você usar apenas uma única abordagem, encontrará apenas os bugs que essa técnica pode detectar... e nada além disso.

Paradoxo do pesticida: cada método usado para prevenir ou encontrar bugs deixa um resíduo de falhas mais sutis, contra as quais esses métodos são ineficazes.

Bugs acontecem em alguns lugares mais do que em outros.

Os bugs não estão distribuídos uniformemente. Empiricamente, sabemos que **alguns componentes apresentam mais falhas do que outros.** Por exemplo, um **módulo de Pagamento** pode exigir testes mais rigorosos do que um **módulo de Marketing.**

Schröter et al. (2006) observaram que **71% dos arquivos que importavam pacotes do compilador precisaram de correções posteriores.** Isso indica que **certos arquivos são mais propensos a defeitos** do que outros dentro do sistema.

Como desenvolvedor, é essencial observar e aprender com seu próprio sistema. Além do código-fonte, **outros dados podem ajudar a definir a priorização dos testes.**

Predicting Component Failures at Design Time

Adrian Schröter
Saarland University
Saarbrücken, Germany
adrian@st.cs.uni-sb.de

Thomas Zimmermann
Saarland University
Saarbrücken, Germany
tz@acm.org

Andreas Zeller
Saarland University
Saarbrücken, Germany
zeller@acm.org

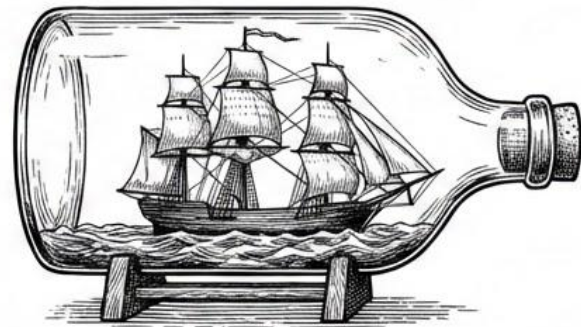
O contexto é a chave

O contexto desempenha um papel fundamental na definição dos casos de teste.



Criar testes para um aplicativo móvel é muito diferente de criar testes para uma aplicação web ou um software usado em um foguete.

Em outras palavras, o teste de software depende do contexto.



Então... como eu monto uma estratégia?

uma **estratégia de teste** deve ser construída de forma **contextual, iterativa e sociotécnica**, observando continuamente as condições que tornam o teste viável, valioso e sustentável naquele projeto.



Comece pelo contexto do projeto.



Distribua o esforço entre camadas complementares.



Avalie riscos, infraestrutura, cultura e recursos.



Materialize a estratégia em testes, exemplos e processos.



Escolha práticas viáveis e percebidas como valiosas.



Revise continuamente, porque a estratégia emerge e evolui com o projeto.

Então... como eu monto uma estratégia?



As decisões de teste dependem da interdependência de condições

como complexidade do projeto, infraestrutura de teste e cultura de teste.



A decisão de usar ou não certas práticas é construída caso a caso

ligada ao projeto concreto, e suas causas não são generalizáveis de forma simples.



Pequenas diferenças no contexto podem gerar grandes diferenças no efeito de uma abordagem de

Por isso, copiar mecanicamente uma estratégia de outro projeto é arriscado.



Estratégias de teste emergem de um processo recursivo

ao testar, a equipe altera infraestrutura, artefatos, cultura e expectativas, o que muda as decisões futuras.



Desenvolvedores tendem a testar quando as condições permitem que o teste seja percebido como eficiente e valioso para o projeto e para quem o executa.

Comece pelo contexto, não pelo modelo

Antes de decidir 'pirâmide', 'árvore' ou qualquer outra heurística, a equipe precisa entender:

✓ complexidade do sistema;

✓ riscos mais relevantes;

✓ restrições de tempo e recursos;

✓ arquitetura e dependências;

✓ maturidade da infraestrutura;

✓ cultura e experiência da equipe.

Identifique o que torna o teste viável

Infraestrutura, exemplos existentes, artefatos de teste e facilidade de uso influenciam muito a adoção de práticas.

Então, montar uma estratégia envolve perguntar:



temos ambiente de teste?



temos dados, fixtures, dublês, pipelines?



já existem exemplos reaproveitáveis?



escrever e rodar testes é algo fácil ou penoso?

Torne explícito o valor do teste

Uma boa estratégia deixa claro os pilares do valor:

Mitigação de Risco



Que risco cada teste mitiga.

Feedback Gerado



Que feedback ele oferece.

Custo Evitado



Que custo ele evita.

Por que Investir?



Por que vale a pena investir nele.

Testing efficacy: o sentimento de que o esforço de teste consegue produzir algo desejado, útil e valioso.

Em termos práticos (O Ciclo de Adoção):

Quando parece **ÚTIL**,
ACESSÍVEL e **VALIOSO**



O teste tende a
CRESCER.

Quando parece **CARO**, **INÚTIL** e **PENOSO**



O teste tende a ser
EVITADO.

Distribua o esforço de modo coerente com o sistema

Combinar níveis e tipos de teste para equilibrar:

Aqui entra o conhecimento clássico de teste de software: combinar níveis e tipos de teste para equilibrar custo, velocidade de feedback, diagnóstico e confiança.



Inscriva a estratégia em artefatos e práticas

A estratégia emerge também de **testing signatures** e **testing echoes**:



Testing Signatures

traços materiais de como o projeto testa, como código de teste, documentação e pipelines;

- artefatos como testes, documentação e pipelines mostram “como se testa aqui”;



Testing Echoes

conversas, discussões, posts, revisões e outros impulsos que carregam perspectivas sobre teste

- conversas, revisões e discussões espalham perspectivas sobre o teste.

Então, uma estratégia robusta precisa aparecer em:

✓ suíte de testes existente;

✓ exemplos de testes bem escritos;

✓ code review;

✓ Definition of Done, Definition of Ready;

✓ documentação leve;

✓ CI/CD.

Preserve autonomia, mas dê direção

Uma estratégia saudável não é só impor cobertura mínima. Ela precisa:

- orientar prioridades;
- apoiar boas decisões;
- evitar burocracia que produza “teste para cumprir regra”.



orientar
prioridades;



apoiar boas
decisões;



evitar burocracia
que produza “teste
para cumprir regra”.

REVISE CONTINUAMENTE A ESTRATÉGIA

Como as decisões de hoje mudam as condições de amanhã, a estratégia precisa ser revista de forma contínua.



DEFEITOS RECORRENTES

Pedem mais testes em certo ponto.



FRAGILIDADE EXCESSIVA

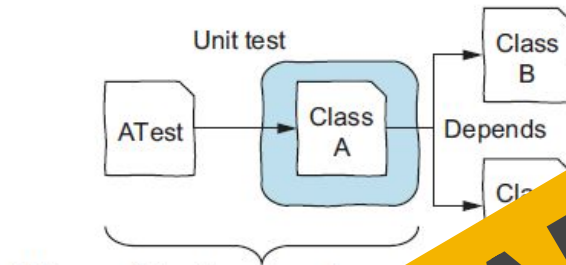
Pede redistribuição do portfólio.



NOVA INFRAESTRUTURA

Viabiliza práticas antes inviáveis.

Teste de Unidade



When unit testing class A, our goal is to test it as isolated as possible from the rest of the system. To do this, we simulate the behavior of the classes it depends on.

Figure 1.5 Unit testing. Our goal is to test one unit of the system that is as isolated as possible from the rest of the system.

- É a fase do teste em que se testam as menores unidades desenvolvidas. O universo alvo desse tipo de teste são métodos dos objetos ou mesmo pequenos trechos de código
- Objetivo: encontrar falhas de funcionamento dentro de uma pequena parte do sistema funcionando independentemente do todo